

Summary

The primary objective of asymptotic analysis is to evaluate how the resource requirements of an algorithm typically time grow as the input size (n) increases toward infinity. The core takeaways from this analysis include:

- **Function Comparison Hierarchy:** There is a defined order of growth across function types, transitioning from logarithmic to polynomial, exponential, and finally factorial/combinatorial growth.
- **Logarithmic Comparison Method:** Taking the logarithm of functions is an effective strategy for comparing growth rates, though constants must be treated with precision during this process.
- **Asymptotic Notations:** Big-O (O), Omega (Ω), and Theta (Θ) serve distinct roles in defining upper, lower, and exact bounds for growth rates.
- **Logical Application:** In algorithmic contexts, Big-O is typically used to define the "Worst Case" (an upper limit), while Omega defines the "Best Case" (a lower limit).
- **Loop Analysis:** The complexity of iterative loops is determined by the update rule; incremental updates generally result in linear complexity ($O(n)$), while multiplicative updates result in logarithmic complexity ($O(\log n)$).

1. Techniques for Function Comparison

1.1 The Logarithmic Method

To compare complex functions like (2^n) and (3^n), one can take the logarithm of both sides:

- **Example:** ($\log(2^n) = n \log 2$) and ($\log(3^n) = n \log 3$).
- **Observation:** While constants are often ignored in asymptotic notation, they are critical when comparing growth rates using logs. Since ($\log 2 < \log 3$), it is confirmed that (2^n) grows slower than (3^n).

1.2 Hierarchy of Growth Rates

Functions can be arranged in non-decreasing order of growth. A key principle is that Polynomial growth always exceeds Logarithmic growth, and Exponential growth is exceeded by Factorial growth.

Growth Category	Example Functions (In Increasing Order)
Decreasing	($\frac{100}{n}$)
Constant	10

Logarithmic	$(\log \log n < \log n < (\log n)^2)$
Polynomial/Sub-linear	$(n^{1/10} < \sqrt{n} < n < n \log \log n < n \log n < n^{3/2} < n^2)$
Exponential	$(2^n < e^n < 3^n)$
Factorial/Super-Exp	$(n! < n^n)$

Key Insight: In growth analysis, constants do not change the complexity class. For example, $(n \log n)$ and $(100 n \log n)$ have the same asymptotic growth.

2. Asymptotic Notations and Logical Significance

2.1 Formal Definitions

- Big-O (O): $(f(n) = O(g(n)))$ implies the growth of $(g(n))$ is greater than or equal to $(f(n))$. It sets an upper bound.
- Omega (Ω): $(f(n) = \Omega(g(n)))$ implies the growth of $(g(n))$ is less than or equal to $(f(n))$. It sets a lower bound.
- Theta (Θ): $(f(n) = \Theta(g(n)))$ implies the growth of $(f(n))$ and $(g(n))$ are exactly the same. This is only possible if both $(f(n) = O(g(n)))$ and $(f(n) = \Omega(g(n)))$ are true.

2.2 Logical Context in Algorithms

- Worst Case (Big-O): Used to determine the maximum risk. It guarantees that the number of operations will never exceed this bound. (Example: Searching an array and only finding the element at the last position or not at all).
- Best Case (Omega): Used to determine the minimum operations required. It guarantees that the complexity will never be less than this bound. (Example: Finding a search element at the very first index of an array).
- Average Case (Theta): Generally used for algorithms where the best and worst cases are consistent or to describe the exact growth rate.

3. Algorithm Categorization

Algorithms are categorized based on their structure, which dictates their analysis method:

1. "Children's" Algorithms (Constant Time): Algorithms with no loops and no recursion. These perform a fixed number of primitive operations (addition, multiplication, jumps) regardless of (n) . Their complexity is $(O(1))$.

2. Iterative Algorithms: These contain loops. Complexity is determined by the number of times the loop body executes.
3. Recursive Algorithms: These call themselves. Analysis requires solving recurrence relations (to be covered in future sessions).

4. Iterative Loop Analysis

The analysis of loops focuses on the "update rule" of the iterator (i).

4.1 Incremental Loops ($O(n)$)

If a loop increments by a constant (e.g., $(i = i + 1)$ or $(i = i + 2)$), the number of executions is proportional to (n) .

- Example: For $(i = 1)$ to (n) with $(i = i + 2)$, the exact number of executions is the ceiling of $(n/2)$. Asymptotically, this is $(O(n))$ because the constant $(1/2)$ is ignored.

4.2 Multiplicative Loops ($O(\log n)$)

If a loop iterator doubles (or triples) each iteration, the growth of (i) is exponential, meaning it reaches (n) very quickly.

- Mathematical Derivation for $(i = i \times 2)$:
 - Initial value: $(i = 1)$
 - Values of (i) : $1, 2, 4, 8, (\dots), (2^k)$
 - The loop terminates when $(2^k > n)$.
 - Solving for (k) : $(k \approx \log_2 n)$.
 - Total iterations: $(\log_2 n + 1)$, which simplifies to $(O(\log n))$.
- Base of Logarithm: If the update is $(i = i \times 3)$, the complexity is $(\log_3 n)$. However, in asymptotic notation, the base is often omitted as it is a constant factor, resulting in $(O(\log n))$.

5. Critical Mathematical Formulas for Analysis

When evaluating the complexity of nested structures or sums, the following mathematical identities are applied:

- Sum of first n integers:

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} \approx \Theta(n^2)$$
- Sum of squares:

$$\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6} \approx \Theta(n^3)$$

$$\sum i^2 = \frac{n(n+1)(2n+1)}{6} \approx \Theta(n^3)$$

• Sum of cubes:

$$\sum i^3 = \left[\frac{n(n+1)}{2} \right]^2 \approx \Theta(n^4)$$

6. Practical Conclusion on Growth

When evaluating statements like ($f(n) = O(g(n))$), one must determine which function dominates as (n) becomes large. As demonstrated in GATE-level problems, even complex-looking functions (like ($n \log \log n$) vs ($n^{3/2}$)) can be simplified by removing common factors to reveal the underlying polynomial vs. logarithmic relationship. Polynomial growth will consistently outpace logarithmic growth in the long run.